

times for a majority of Intel devices makes them a great choice for object detection programs and a majority of deep learning cases.

```
# Create stub
host, port = FLAGS.server.split(':')
channel = implementations.insecure_channel(host, int(port))
stub = prediction_service_pb2.beta_create_PredictionService_
stub(channel)

# Create prediction request object
request = predict_pb2.PredictRequest()
# Specify model name (must be the same as when the TensorFlow serving
was started)
request.model_spec.name = 'obj_det'
# Initialize prediction
# Specify signature name (should be the same as specified when exporting
model)
request.model_spec.signature_name = "detection_signature"
request.inputs['inputs'].CopyFrom(
tf.contrib.util.make_tensor_proto({FLAGS.input_image}))
# Call the prediction server
result = stub.Predict(request, 10.0) # 10 secs timeout
# Plot boxes on the input image
category_index = load_label_map(FLAGS.path_to_labels)
boxes = result.outputs['detection_boxes'].float_val
classes = result.outputs['detection_classes'].float_val
scores = result.outputs['detection_scores'].float_val
image_vis = vis_util.visualize_boxes_and_labels_on_image_array(
    FLAGS.input_image,
    np.reshape(boxes,[100,4]),
    np.squeeze(classes).astype(np.int32),
    np.squeeze(scores),
    category_index,
    use_normalized_coordinates=True,
    line_thickness=8)
# Save inference to disk
scipy.misc.imsave('%s.jpg'%(FLAGS.input_image), image_vis)
```

Inference docker commands

Run the following commands to establish a link between both the